Penerapan Algoritma Genetika dan *Binary Particle Swarm Optimization* pada Optimasi Alokasi *Dieset* di *Press Shop* BMW

Firsta Sucahyo
Program Studi Artificial Intelligence
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
Bandung, Indonesia
28426004@mahasiswa.itb.ac.id

Muhamad Nazih Najmudin
Program Studi Artificial Intelligence
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
Bandung, Indonesia
13523144@mahasiswa.itb.ac.id

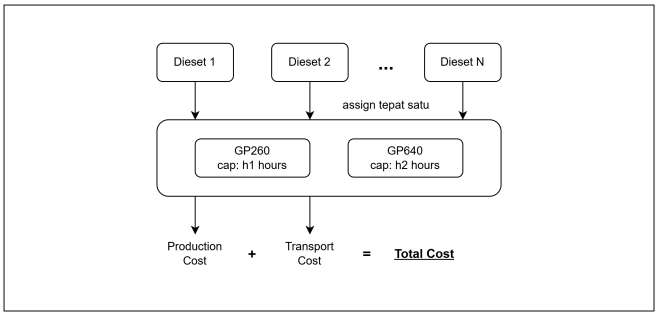
***Ringkasan*—Makalah ini merancang dan mengevaluasi *problem solving agent* untuk persoalan alokasi *dieset* ke mesin pres pada *press shop* BMW menggunakan *dataset* industri *Path Matters*, yang pada penelitian sebelumnya diselesaikan dengan *quantum annealing* dan LR-QAOA. Persoalan dimodelkan sebagai *generalized assignment problem* dengan dua mesin. Solusi direpresentasikan sebagai vektor biner sepanjang jumlah *dieset* fleksibel, sedangkan fungsi *fitness* didefinisikan sebagai total biaya ditambah penalti pelanggaran kapasitas. Tiga algoritma dibandingkan, yaitu algoritma genetika (GA), *Binary Particle Swarm Optimization* (BPSO), dan *random search* sebagai *baseline*, pada enam instans berukuran 3 hingga 19 *dieset*. Setiap konfigurasi dijalankan 30 kali dengan *seed* berbeda dan anggaran yang sama, yaitu 5.000 evaluasi *fitness*, sehingga diperoleh 3.240 *run*. Seluruh *run* menghasilkan solusi layak dengan *gap* di bawah 1% terhadap optimum *brute-force*. Konfigurasi terbaik GA (*rank selection*) dan BPSO (ukuran *swarm* 20) mencapai *gap* rata-rata 0,0024% hingga 0,0338% pada instans 13 hingga 19. Kedua algoritma secara signifikan lebih baik daripada *random search* pada instans 16 hingga 19 berdasarkan uji Mann-Whitney U dengan nilai p di bawah 0,001. Perbedaan GA dan BPSO tidak signifikan pada seluruh instans, tetapi waktu eksekusi BPSO sekitar 4,5 kali lebih cepat. Analisis sensitivitas menunjukkan bahwa ukuran populasi atau *swarm* merupakan parameter yang paling berpengaruh pada kedua algoritma.**

***Kata Kunci*—algoritma genetika, *binary particle swarm optimization*, *generalized assignment problem*, perencanaan produksi, metaheuristik**

I. PENDAHULUAN

A. Latar Belakang

Dalam produksi kendaraan, komponen bodi seperti pintu, *side-frame*, dan panel dibentuk dari lembaran logam di *press shop*. Setiap komponen dibentuk oleh *dieset*, yaitu perangkat cetak yang dipasang pada salah satu mesin pres. Pemasangan *dieset* yang sama pada mesin yang berbeda menimbulkan biaya yang berbeda karena adanya perbedaan efisiensi mesin (biaya produksi) dan jarak menuju *assembly plant* (biaya transportasi). Di sisi lain, setiap mesin memiliki kapasitas jam kerja yang terbatas dalam satu periode produksi. Oleh karena itu, perencana produksi perlu menentukan alokasi *dieset* ke mesin yang meminimalkan total biaya tanpa melampaui kapasitas mesin. Ilustrasi persoalan ini ditunjukkan pada Gambar 1.



Gambar 1. Ilustrasi persoalan alokasi *dieset*. Setiap *dieset* dialokasikan ke tepat satu mesin pres (GP260 atau GP640) dengan kapasitas jam kerja terbatas; total biaya merupakan jumlah biaya produksi dan biaya transportasi.

Alokasi *dieset* ke mesin dengan biaya untuk setiap pasangan *dieset*-mesin dan kapasitas untuk setiap mesin merupakan bentuk *generalized assignment problem* (GAP). GAP termasuk persoalan NP-*hard* yang telah lama dikaji, antara lain dengan metode *branch and bound* [1]. Chu dan Beasley [2] menunjukkan bahwa GA dengan representasi bilangan bulat, yaitu satu gen untuk setiap *item* yang berisi indeks agen, disertai penanganan kendala kapasitas, efektif untuk menyelesaikan GAP.

B. Penelitian Terkait

Schmidbauer dkk. [3] mempublikasikan persoalan alokasi *dieset* sebagai *dataset* terbuka yang bersumber dari data produksi BMW. Persoalan tersebut diselesaikan dengan *quantum annealing* (D-Wave), LR-QAOA (IBM), dan *simulated annealing* setelah ditransformasikan dari formulasi *binary integer linear program* (BILP) ke bentuk QUBO (*Quadratic Unconstrained Binary Optimization*). Transformasi ini memerlukan variabel *slack* dan bobot penalti sehingga instans dengan 19 *dieset* berkembang menjadi persoalan berukuran 60 *qubit*. Hasilnya menunjukkan bahwa *quantum annealing* pada D-Wave

secara konsisten menghasilkan solusi yang mendekati optimal, sedangkan LR-QAOA pada perangkat IBM belum mencapai kualitas yang sebanding, dan bahwa strategi penalti sangat menentukan kualitas solusi. Sebagai pembanding, makalah tersebut menggunakan *simulated annealing* dan solusi terbaik dari 1.000 *bitstring* acak, sedangkan nilai optimum acuan dihitung menggunakan *solver* SCIP pada model BILP [4].

Safi dkk. [5] melakukan studi serupa pada persoalan *job-shop scheduling* industri dari Siemens dan menyimpulkan bahwa formulasi persoalan dan pemilihan *solver* lebih menentukan kualitas solusi daripada jenis perangkat keras. Thelen dan Mauerer [6] menyoroti *overhead* yang timbul ketika kendala ditransformasikan ke format masukan algoritma kuantum. Ketiga studi tersebut menunjukkan bahwa formulasi persoalan merupakan bagian penting dalam perancangan *solver*. Berbeda dengan pendekatan kuantum, metaheuristik klasik tidak memerlukan transformasi ke QUBO karena kendala dapat ditangani secara langsung melalui representasi solusi dan fungsi *fitness*. Namun, kinerja metaheuristik klasik pada *dataset* ini belum dievaluasi secara sistematis.

C. Tujuan dan Kontribusi

Berdasarkan latar belakang tersebut, makalah ini menjawab tiga pertanyaan penelitian berikut.

1) Bagaimana memodelkan persoalan alokasi *dieset* sebagai persoalan optimasi, yang mencakup keadaan (*state*), representasi, fungsi biaya, dan operator, sehingga dapat diselesaikan oleh GA dan algoritma kecerdasan kawanan (*swarm intelligence*)?
2) Bagaimana kualitas solusi GA dan BPSO dibandingkan dengan *baseline random search* pada anggaran evaluasi yang sama, diukur dari *gap* terhadap nilai optimum?
3) Parameter dan operator apa yang paling berpengaruh terhadap kualitas solusi setiap algoritma?

Kontribusi makalah ini adalah sebagai berikut.

- Formulasi persoalan sebagai pencarian biner dengan representasi satu gen per *dieset* fleksibel dan fungsi penalti kuadratik dengan suku indikator, yang menjamin solusi layak selalu lebih baik daripada solusi tak layak tanpa memerlukan operator perbaikan.
- Implementasi GA dan BPSO yang dikembangkan secara mandiri dalam kerangka yang sama sehingga jumlah evaluasi dihitung dengan cara yang identik untuk seluruh algoritma.
- Rancangan eksperimen dengan 30 *seed* per konfigurasi dan anggaran evaluasi yang seragam, analisis sensitivitas terhadap 15 konfigurasi GA dan BPSO, serta uji signifikansi statistik antaralgoritma.
- Paket eksperimen yang seluruh tabel dan gambarnya dapat direproduksi dengan satu perintah.

Bagian II menguraikan metode penelitian, yang meliputi data, formulasi persoalan, algoritma, dan desain eksperimen. Bagian III menyajikan hasil eksperimen, Bagian IV membahas interpretasi dan keterbatasannya, dan Bagian V memuat kesimpulan serta arah penelitian selanjutnya.

II. METODE

A. Data dan Instans

Dataset diperoleh dari Zenodo [4] sebagai berkas pendukung makalah [3]. *Dataset* ini terdiri atas enam instans yang bersumber dari data produksi BMW dengan dua mesin pres, yaitu GP260 dan GP640. Setiap instans tersedia dalam tiga varian QUBO (`raw`, `rounded`, dan `scaled`) yang hanya berbeda pada bobot penalti, sedangkan *field* `model` yang memuat data biaya, beban kerja, dan kapasitas bernilai sama pada ketiga varian. Penelitian ini hanya menggunakan *field* `model`.

Sebagian *dieset* hanya dapat dipasang pada satu mesin (*dieset* tetap). *Dieset* tetap tidak diperlakukan sebagai variabel keputusan, tetapi beban kerjanya mengurangi kapasitas mesin yang tersedia. Tahap pra-pemrosesan mengekstraksi *dieset* fleksibel, menghitung kapasitas sisa setiap mesin, dan menghitung optimum *brute-force* melalui enumerasi seluruh 2^n kombinasi. Ringkasan setiap instans disajikan pada Tabel I.

Optimum SCIP dihitung pada model dengan kapasitas yang dibulatkan ke bawah dan beban kerja yang dibulatkan ke atas. Akibat pembulatan tersebut, solusi SCIP pada instans 16, 18, dan 19 bukan merupakan optimum bagi model tanpa pembulatan. Oleh karena itu, *gap* dalam penelitian ini dihitung terhadap optimum *brute-force*, sedangkan *gap* solusi SCIP pada model tanpa pembulatan (0,041%, 0,016%, dan 0,046%) ditampilkan sebagai garis pembanding.

B. Formulasi Persoalan

1) Model Matematis: Misalkan T adalah himpunan *dieset* fleksibel, M adalah himpunan mesin, c_{tm} adalah biaya alokasi *dieset* t ke mesin m, w_{tm} adalah beban kerja dalam satuan jam, dan h_m adalah kapasitas sisa mesin m. Dengan variabel keputusan $x_{tm} \in \{0,1\}$, persoalan dirumuskan sebagai

$$\min_{x} \quad \sum_{t\in T}\sum_{m\in M} c_{tm}\, x_{tm} \tag{1}$$

$$\text{s.t.} \quad \sum_{t\in T} w_{tm}\, x_{tm} \le h_m, \qquad \forall m \in M, \tag{2}$$

$$\sum_{m\in M} x_{tm} = 1, \qquad \forall t \in T, \tag{3}$$

dengan (2) sebagai kendala kapasitas dan (3) sebagai kendala penugasan.

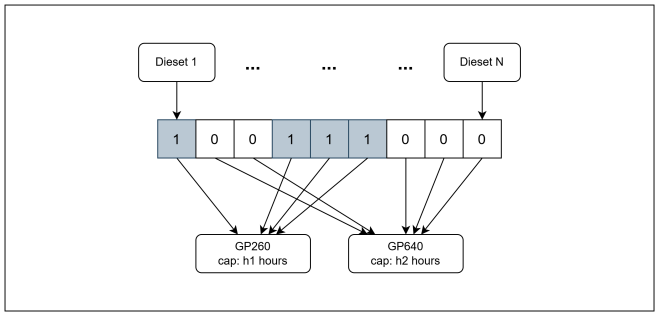
2) Keadaan, Representasi, dan Operator: Persoalan diformulasikan sebagai pencarian keadaan lengkap (*complete-state search*), yaitu setiap keadaan merupakan satu alokasi lengkap bagi seluruh *dieset* fleksibel. Pada keenam instans, setiap *dieset* fleksibel memiliki tepat dua pilihan mesin sehingga keadaan dapat direpresentasikan sebagai vektor biner

$$\mathbf{g} \in \{0,1\}^n, \quad g_i = j \iff \textit{dieset } i \text{ ke pilihan mesin ke-}j. \tag{4}$$

Representasi (4), yang diilustrasikan pada Gambar 2, mengikuti pendekatan satu gen per *item* dari Chu dan Beasley [2] dan menjamin kendala penugasan (3) selalu terpenuhi karena setiap gen selalu memilih tepat satu mesin. Representasi yang

Tabel I
UKURAN INSTANS DAN NILAI OPTIMUM ACUAN (n MENYATAKAN JUMLAH *Dieset* FLEKSIBEL)

n	*Dieset* tetap	Ruang pencarian	Solusi layak	Optimum SCIP [3]	Optimum *brute-force*
3	13	8	1 (12,5%)	443.483,21	443.483,21
9	15	512	179 (35,0%)	2.249.378,78	2.249.378,78
13	19	8.192	2.951 (36,0%)	5.223.596,12	5.223.596,12
16	19	65.536	27.821 (42,5%)	5.803.504,82	5.801.121,21
18	22	262.144	114.795 (43,8%)	5.963.876,62	5.962.911,99
19	28	524.288	222.128 (42,4%)	6.115.761,14	6.112.960,59



Gambar 2. Representasi solusi. Setiap gen mewakili satu *dieset* fleksibel, dan nilainya menentukan mesin tempat *dieset* tersebut dialokasikan.

sama digunakan oleh GA, BPSO, dan *random search* sehingga perbedaan hasil hanya bersumber dari cara setiap algoritma menjelajahi ruang pencarian. Keadaan awal dibangkitkan secara acak. Operator yang mengubah keadaan adalah *crossover* dan mutasi pada GA, serta pembaruan kecepatan dan fungsi transfer pada BPSO, sebagaimana diuraikan pada Bagian II-C.

3) Fungsi Biaya dan Penanganan Kendala: Kendala kapasitas (2) ditangani dengan fungsi penalti. Fungsi *fitness* yang diminimalkan adalah

$$f(\mathbf{g}) = \sum_{i=1}^{n} c_{i,g_i} + \lambda\, P(\mathbf{g}), \tag{5}$$

dengan suku penalti

$$P(\mathbf{g}) = \sum_{m \in M} \max\big(0, L_m(\mathbf{g}) - h_m\big)^2 + \mathbb{1}\big[\exists m : L_m(\mathbf{g}) > h_m\big], \tag{6}$$

$c_{i,j}$ menyatakan biaya *dieset* i pada pilihan mesin ke-j, $L_m(\mathbf{g})$ menyatakan total beban kerja pada mesin m, dan $\lambda = \sum_i \max_j c_{i,j}$ menyatakan batas atas total biaya. Suku kuadrat memberikan penalti yang sebanding dengan besarnya pelanggaran sehingga algoritma tetap memperoleh arah perbaikan ketika berada di daerah tak layak. Suku indikator menjamin bahwa setiap solusi tak layak memiliki nilai *fitness* lebih besar daripada λ, sedangkan nilai *fitness* solusi layak tidak melebihi λ. Dengan demikian, solusi layak selalu lebih baik daripada solusi tak layak. Solusi antara selama proses pencarian diperbolehkan tidak layak, tetapi solusi akhir diverifikasi kembali terhadap kendala kapasitas. Penelitian ini tidak menggunakan operator perbaikan (*repair operator*).

C. Algoritma

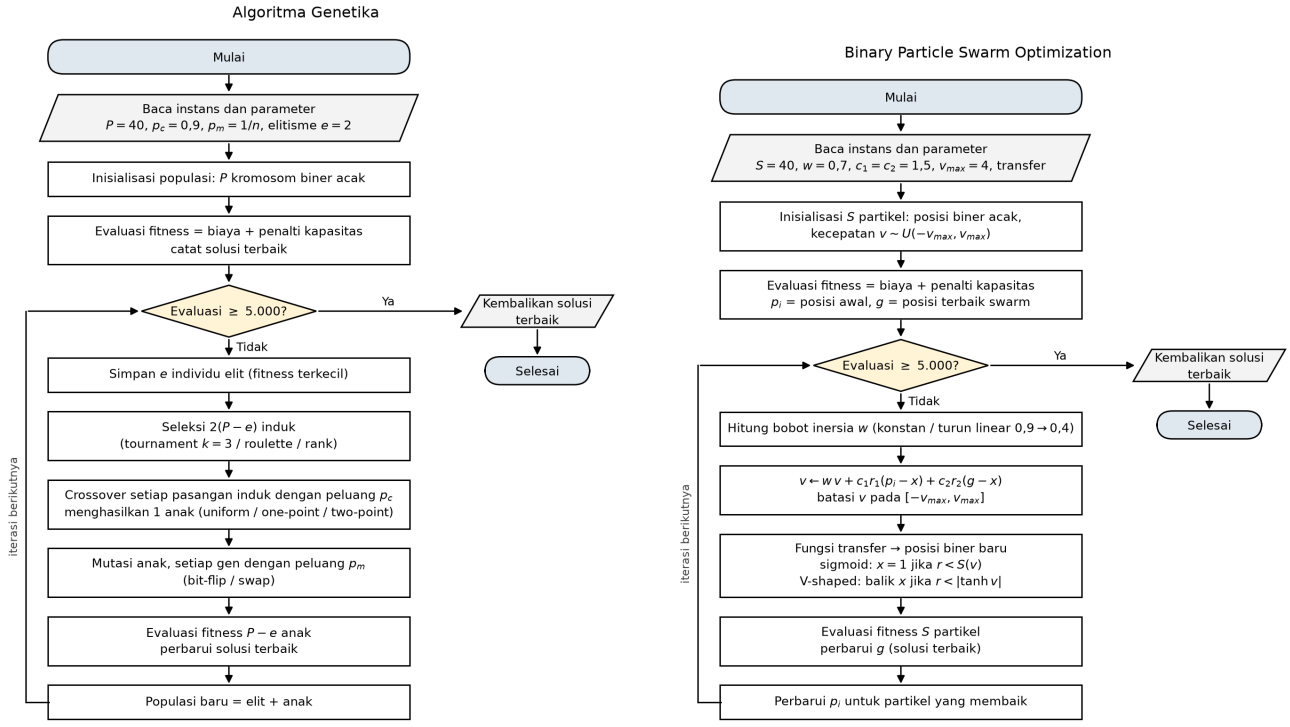
Ketiga algoritma diturunkan dari satu kelas dasar yang menangani evaluasi *fitness*, penghitungan jumlah evaluasi, pencatatan solusi terbaik, dan kriteria penghentian, sedangkan setiap algoritma hanya mengimplementasikan tahap inisialisasi dan satu iterasi pencarian. Struktur ini menjamin bahwa jumlah evaluasi dihitung dengan cara yang sama untuk seluruh algoritma. Alur GA dan BPSO ditunjukkan pada Gambar 3.

1) Algoritma Genetika: Algoritma genetika [7] mengadaptasi prinsip seleksi alam, yaitu populasi solusi berevolusi melalui proses seleksi, *crossover*, dan mutasi. GA yang digunakan merupakan GA generasional dengan elitisme (Gambar 3a). Pada setiap generasi, e individu terbaik dipertahankan, kemudian dipilih $2(P-e)$ induk. Setiap pasangan induk menghasilkan satu anak melalui *crossover* dengan peluang p_c, lalu anak tersebut dimutasi. Populasi generasi berikutnya terdiri atas individu elit dan anak-anak yang dihasilkan. Individu elit tidak dievaluasi ulang sehingga setiap generasi menggunakan $P-e$ evaluasi. Operator yang diimplementasikan merupakan pilihan standar yang dibahas oleh Goldberg [7], yaitu sebagai berikut.

- Seleksi: *tournament* dengan ukuran k; *roulette wheel* dengan bobot $f_{\max} - f$ karena *fitness* diminimalkan; dan *rank selection* dengan bobot linear dari P untuk individu terbaik hingga 1 untuk individu terburuk.
- *Crossover*: satu titik (*one-point*), dua titik (*two-point*), dan *uniform*, yaitu setiap gen diambil dari salah satu induk dengan peluang 0,5.
- Mutasi: *bit-flip*, yaitu memindahkan *dieset* ke mesin lain, dan *swap*, yaitu menukar nilai dua gen.

Nilai bawaan setiap parameter beserta dasar penetapannya disajikan pada Tabel II.

2) Binary Particle Swarm Optimization: Particle Swarm Optimization (PSO) [8] mengadaptasi perilaku kawanan, yaitu setiap partikel bergerak berdasarkan posisi terbaik yang pernah dicapainya dan posisi terbaik seluruh *swarm*. Kennedy dan Eberhart [9] mengembangkan PSO untuk ruang biner (BPSO) dengan mengubah kecepatan menjadi peluang suatu bit bernilai 1 melalui fungsi sigmoid. Shi dan Eberhart [10] memperkenalkan bobot inersia untuk mengatur keseimbangan antara eksplorasi dan eksploitasi, sedangkan Mirjalili dan Lewis [11] menunjukkan bahwa fungsi transfer berbentuk V (*V-shaped*), yang menyatakan peluang suatu bit dibalik, dapat mengatasi kelemahan fungsi sigmoid ketika nilai kecepatan besar.



(a) (b)

Gambar 3. Diagram alir (a) algoritma genetika dan (b) *Binary Particle Swarm Optimization*.

Tabel II
PARAMETER BAWAAN GA

Parameter	Nilai bawaan	Dasar penetapan
Ukuran populasi P	40	sama dengan ukuran *swarm* BPSO; menghasilkan 131 generasi dalam 5.000 evaluasi
Seleksi	*tournament*, $k = 3$	tekanan seleksi moderat dengan ukuran turnamen kecil
Crossover	*uniform*, $p_c = 0{,}9$	urutan *dieset* pada kromosom tidak bermakna sehingga gen bersebelahan tidak perlu diwariskan bersama; p_c tinggi lazim digunakan [7]
Mutasi	*bit-flip*, $p_m = 1/n$	setiap *dieset* hanya memiliki dua pilihan mesin; rata-rata satu gen berubah pada setiap anak
Elitisme e	2	solusi terbaik tidak hilang, tetapi tidak mendominasi populasi

PSO dipilih dibandingkan dengan *Ant Colony Optimization* (ACO) berdasarkan struktur persoalan. ACO sesuai untuk persoalan yang solusinya dibangun secara bertahap pada suatu graf, seperti *traveling salesman problem*, sedangkan persoalan ini tidak memiliki urutan konstruksi yang bermakna. Ruang pencarian persoalan ini berupa *hypercube* biner berdimensi n, yang merupakan ruang asli BPSO [9]. Setiap dimensi partikel mewakili satu *dieset*, dan kecepatan menentukan peluang *dieset* tersebut dialokasikan ke mesin kedua. Representasi, fungsi *fitness*, dan penalti yang digunakan identik dengan GA sehingga operator perbaikan tidak diperlukan. Dimensi persoalan juga relatif kecil (3 hingga 19) sehingga kelemahan BPSO pada dimensi tinggi tidak muncul. Selain itu, solusi optimum terletak tepat di batas kapasitas salah satu mesin, dengan sisa kapasitas GP640 pada solusi optimum hanya 1,5 hingga 5,4 jam. Kondisi ini menuntut kendali yang baik atas eksplorasi dan eksploitasi, yang pada BPSO diatur melalui bobot inersia, $v_{\max}$, dan fungsi transfer.

Kecepatan partikel i pada dimensi d diperbarui dengan persamaan

$$v_{id} \leftarrow w\, v_{id} + c_1 r_1 (p_{id} - x_{id}) + c_2 r_2 (g_d - x_{id}), \qquad (7)$$

dengan $r_1, r_2 \sim U(0,1)$, kemudian dibatasi pada selang $[-v_{\max}, v_{\max}]$. Notasi p_i menyatakan posisi terbaik partikel i, sedangkan g menyatakan posisi terbaik yang pernah ditemukan oleh *swarm*. Posisi biner yang baru ditentukan oleh salah satu fungsi transfer berikut, dengan $r \sim U(0,1)$:

- sigmoid (*S-shaped*) [9]: $x_{id} = 1$ jika $r < 1/(1 + e^{-v_{id}})$;
- *V-shaped* [11]: nilai x_{id} dibalik jika $r < |\tanh(v_{id})|$.

Bobot inersia w dapat bernilai konstan atau menurun secara linear dari w_0 ke w_{end} sepanjang anggaran evaluasi. Alur lengkap BPSO ditunjukkan pada Gambar 3b, sedangkan parameter bawaannya disajikan pada Tabel III.

3) Baseline: Random Search: *Random search* membangkitkan sejumlah kromosom acak pada setiap iterasi tanpa memanfaatkan informasi dari solusi sebelumnya, kemudian menyimpan solusi terbaik yang ditemukan. *Baseline* ini diadaptasi dari referensi acak pada makalah sumber data [3], yang menggunakan solusi terbaik dari 1.000 *bitstring* acak. Dalam penelitian ini, *random search* memperoleh anggaran

Tabel III
PARAMETER BAWAAN BPSO

Parameter	Nilai bawaan	Dasar penetapan
Ukuran *swarm* S	40	sama dengan ukuran populasi GA; menghasilkan 124 iterasi dalam 5.000 evaluasi
Fungsi transfer	sigmoid	sesuai dengan BPSO asli [9]
Bobot inersia w	0,7 (konstan)	dalam rentang yang dianjurkan Shi dan Eberhart [10]
c_1, c_2	1,5 dan 1,5	menyeimbangkan tarikan menuju posisi terbaik partikel dan posisi terbaik *swarm*
$v_{\max}$	4	menjaga $S(v)$ di antara 0,018 dan 0,982 sehingga peluang bit tidak jenuh pada 0 atau 1

Tabel IV
KONFIGURASI EKSPERIMEN

Algoritma	Konfigurasi
GA (8)	`default`; seleksi `sel_roulette`, `sel_rank`; *crossover* `cx_one_point`, `cx_two_point`; mutasi `mut_swap`; populasi `pop_20`, `pop_80`
BPSO (7)	`sigmoid` (bawaan); inersia linear 0,9 ke 0,4 `sigmoid_lin`; transfer `vshape`, `vshape_lin`; ukuran *swarm* `sigmoid_s20`, `sigmoid_s80`; `sigmoid_v6` ($v_{\max} = 6$)
Random search (3)	jumlah kromosom per iterasi 20, 40, dan 80

5.000 evaluasi yang sama dengan GA dan BPSO. Karena tidak memanfaatkan struktur persoalan, *random search* berfungsi sebagai batas bawah kinerja: algoritma yang tidak lebih baik daripada *random search* berarti tidak memanfaatkan informasi dari evaluasi sebelumnya secara efektif.

D. Desain Eksperimen

1) Kesetaraan Kondisi Perbandingan: Seluruh algoritma menggunakan representasi, fungsi *fitness*, dan anggaran yang sama, yaitu 5.000 evaluasi *fitness* per *run*, tanpa mekanisme penghentian dini. Kriteria penghentian diperiksa setelah satu iterasi selesai sehingga beberapa konfigurasi menggunakan sedikit lebih banyak evaluasi, yaitu 5.018 untuk GA dengan populasi 40, 5.006 untuk populasi 20, 5.072 untuk populasi 80, serta 5.040 untuk BPSO dengan ukuran *swarm* 80 dan *random search* dengan *batch* 80. Konfigurasi lainnya menggunakan tepat 5.000 evaluasi. Selisih terbesar sebesar 1,4% dinilai tidak memengaruhi kesimpulan.

2) Seed dan Jumlah Run: Setiap konfigurasi dijalankan sebanyak 30 kali pada setiap instans. Daftar 30 *seed* dibangkitkan dari *master seed* 2026, dan daftar yang sama digunakan untuk seluruh konfigurasi. Dengan demikian, total *run* adalah 18 konfigurasi $\times$ 6 instans $\times$ 30 *seed* = 3.240 *run*.

3) Konfigurasi dan Analisis Sensitivitas: Setiap konfigurasi selain konfigurasi bawaan hanya mengubah satu operator atau parameter (*one-factor-at-a-time*) sehingga perbandingan antarkonfigurasi sekaligus berfungsi sebagai analisis sensitivitas. Daftar konfigurasi disajikan pada Tabel IV. Konfigurasi terbaik setiap algoritma ditentukan berdasarkan rata-rata pada enam instans dengan urutan prioritas sebagai berikut: persentase *run* yang menghasilkan solusi layak, persentase *run* yang mendekati optimal (*gap* maksimum 1%), dan *gap* rata-rata terkecil. Konfigurasi terbaik tersebut selanjutnya dibandingkan antaralgoritma.

4) Metrik Evaluasi: Untuk setiap algoritma dan instans, dilaporkan nilai biaya terbaik, rata-rata, simpangan baku, dan terburuk dari 30 *run*, serta *gap* yang didefinisikan sebagai

$$\text{gap} = \frac{\text{cost} - \text{cost}^*}{\text{cost}^*} \times 100\%, \tag{8}$$

dengan cost^* menyatakan optimum *brute-force*. Metrik lain yang digunakan adalah persentase *run* yang menghasilkan solusi layak, persentase *run* yang mencapai optimum, jumlah evaluasi hingga solusi terbaik ditemukan, waktu eksekusi, serta kurva konvergensi berupa rata-rata *gap* solusi terbaik dari 30 *run* terhadap jumlah evaluasi.

5) Uji Signifikansi: Gap akhir dua algoritma dibandingkan pada setiap instans menggunakan uji Mann-Whitney U dua sisi [12], yang dikenal pula sebagai uji Wilcoxon *rank-sum*, dengan tingkat signifikansi $\alpha = 0{,}05$. Uji nonparametrik ini dipilih karena tidak mensyaratkan data berdistribusi normal, sedangkan distribusi *gap* jelas tidak normal, yang ditunjukkan oleh banyaknya *run* dengan *gap* tepat 0. Apabila perbedaan signifikan, algoritma dengan median *gap* yang lebih kecil dinyatakan lebih baik.

6) Reproduksibilitas: Implementasi ditulis dalam bahasa Python 3.12 dengan pustaka NumPy, pandas, SciPy, dan Matplotlib. *Seed*, anggaran evaluasi, dan *hyperparameter* setiap konfigurasi disimpan dalam satu berkas konfigurasi eksperimen. Seluruh tabel dan gambar dapat dibangkitkan ulang dengan satu perintah dalam waktu sekitar 6 menit. Kode sumber, konfigurasi, dan hasil mentah setiap *run* tersedia pada repositori publik [13].

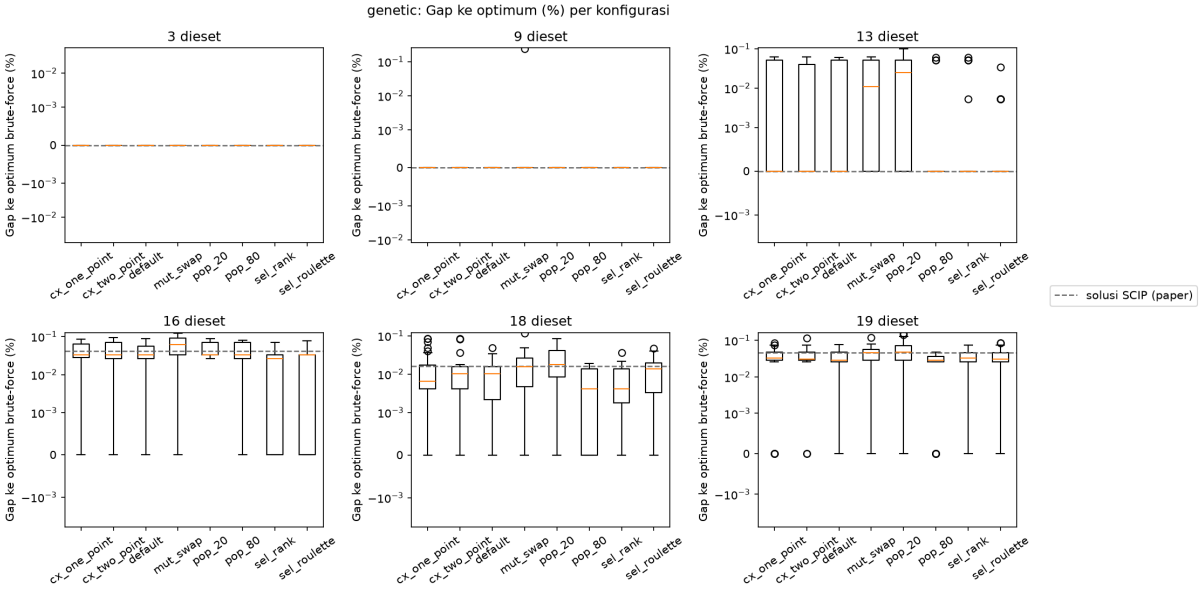
III. HASIL

Seluruh 3.240 *run* menghasilkan solusi layak dengan *gap* di bawah 1% sehingga kriteria *near-optimal* 1% yang digunakan pada makalah [3] tidak dapat membedakan kinerja algoritma. Pada instans 3 dan 9, seluruh konfigurasi selalu mencapai optimum, kecuali satu *run* GA `mut_swap` pada instans 9. Oleh karena itu, penyajian hasil berikut difokuskan pada instans 13 hingga 19.

A. Analisis Sensitivitas GA

Tabel V menyajikan *gap* rata-rata setiap konfigurasi GA, sedangkan Gambar 4 menunjukkan sebaran *gap* dari 30 *run*.

Ukuran populasi merupakan parameter yang paling berpengaruh pada GA. *Gap* rata-rata menurun dari 0,0270% pada populasi 20 menjadi 0,0172% pada populasi 40 dan 0,0128% pada populasi 80. Di antara operator, seleksi memberikan perbaikan terbesar: *rank selection* dan *roulette wheel* menurunkan *gap* konfigurasi bawaan dari 0,0172% menjadi masing-masing 0,0119% dan 0,0139%. *Uniform crossover* menghasilkan kinerja yang lebih baik daripada *one-point* (0,0196%) dan *two-point* (0,0185%). Mutasi *swap* merupakan variasi operator



Gambar 4. Sebaran *gap* dari 30 *run* per konfigurasi GA pada keenam instans. Garis putus-putus menunjukkan *gap* solusi SCIP.

Tabel V
Gap RATA-RATA (%) DARI 30 *Run* PER KONFIGURASI GA, DIURUTKAN BERDASARKAN PERINGKAT

Konfigurasi	n=13	n=16	n=18	n=19	Rerata[a]	Opt.[b]
`sel_rank`	0,0072	0,0220	0,0084	0,0338	0,0119	57,2
`pop_80`	0,0054	0,0367	0,0069	0,0277	0,0128	60,0
`sel_roulette`	0,0020	0,0311	0,0135	0,0366	0,0139	56,1
`default`	0,0190	0,0387	0,0113	0,0343	0,0172	51,7
`cx_two_point`	0,0152	0,0431	0,0145	0,0381	0,0185	50,0
`cx_one_point`	0,0218	0,0408	0,0177	0,0375	0,0196	48,3
`mut_swap`	0,0254	0,0600	0,0204	0,0432	0,0262	43,3
`pop_20`	0,0289	0,0523	0,0268	0,0542	0,0270	43,9

[a]Rata-rata *gap* pada enam instans. [b]Persentase *run* yang mencapai optimum.

Tabel VI
Gap RATA-RATA (%) DARI 30 *Run* PER KONFIGURASI BPSO, DIURUTKAN BERDASARKAN PERINGKAT

Konfigurasi	n=13	n=16	n=18	n=19	Rerata[a]	Opt.[b]
`sigmoid_s20`	0,0024	0,0180	0,0101	0,0308	0,0102	60,6
`sigmoid_v6`	0,0003	0,0246	0,0062	0,0352	0,0111	63,9
`sigmoid_s80`	0,0000	0,0327	0,0103	0,0336	0,0128	57,2
`vshape_lin`	0,0043	0,0281	0,0083	0,0415	0,0137	56,1
`sigmoid_lin`	0,0003	0,0389	0,0116	0,0328	0,0139	57,8
`sigmoid`	0,0033	0,0362	0,0125	0,0376	0,0149	52,8
`vshape`	0,0063	0,0399	0,0117	0,0486	0,0177	50,0

[a]Rata-rata *gap* pada enam instans. [b]Persentase *run* yang mencapai optimum.

dengan kinerja terburuk, dengan *gap* rata-rata 0,0262% dan *gap* maksimum 0,110%.

B. Analisis Sensitivitas BPSO

Tabel VI menyajikan *gap* rata-rata setiap konfigurasi BPSO, sedangkan Gambar 5 menunjukkan sebaran *gap* dari 30 *run*.

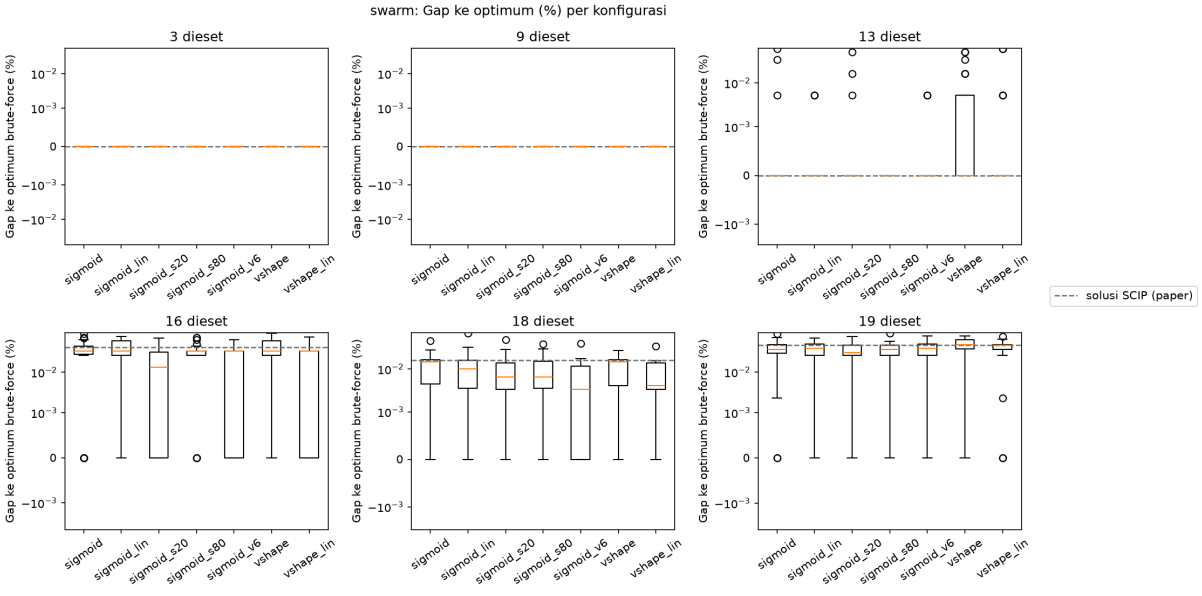
Ukuran *swarm* merupakan parameter yang paling berpengaruh pada BPSO. *Swarm* berukuran 20 menurunkan *gap* konfigurasi bawaan dari 0,0149% menjadi 0,0102%, yang merupakan perbaikan terbesar di antara seluruh variasi. Pengaruh ukuran *swarm* tidak bersifat monoton karena *swarm* berukuran 80 (0,0128%) juga lebih baik daripada *swarm* berukuran 40. Dengan bobot inersia konstan, fungsi transfer sigmoid lebih baik daripada *V-shaped* (0,0149% dibandingkan dengan 0,0177%). Penurunan inersia secara linear memperbaiki *V-shaped* lebih besar (menjadi 0,0137%) daripada sigmoid (menjadi 0,0139%) sehingga kedua fungsi transfer menjadi hampir setara. Nilai $v_{\max} = 6$ menghasilkan persentase pencapaian optimum tertinggi, yaitu 63,9%. Selisih antarkonfigurasi pada peringkat tengah lebih kecil daripada variasi antar-*seed* sehingga urutan peringkat tersebut dapat berubah apabila *seed* yang digunakan berbeda.

C. Perbandingan Antaralgoritma

Konfigurasi terbaik setiap algoritma adalah `sel_rank` untuk GA, `sigmoid_s20` untuk BPSO, dan `batch_20` untuk *random search*. Ketiga konfigurasi *random search* menghasilkan nilai yang identik karena ketiganya membangkitkan urutan 5.000 kromosom acak yang sama dari *seed* yang sama dan hanya berbeda dalam pengelompokan per iterasi. Statistik hasil ketiga algoritma disajikan pada Tabel VII, sedangkan sebaran *gap* dari 30 *run* ditunjukkan pada Gambar 6.

GA dan BPSO mencapai optimum sekurang-kurangnya satu kali pada setiap instans, sedangkan *random search* tidak pernah mencapai optimum pada instans 18 dan 19. Persentase *run* yang mencapai optimum pada instans 13, 16, 18, dan 19 berturut-turut adalah 83,3%, 36,7%, 16,7%, dan 6,7% untuk GA; 90,0%, 50,0%, 10,0%, dan 13,3% untuk BPSO; serta 50,0%, 3,3%, 0%, dan 0% untuk *random search*. Hasil uji signifikansi disajikan pada Tabel VIII.

GA dan BPSO, sebagai dua algoritma terbaik, tidak berbeda secara signifikan pada seluruh instans ($p > 0{,}3$). Kedua algo-



Gambar 5. Sebaran *gap* dari 30 *run* per konfigurasi BPSO pada keenam instans. Garis putus-putus menunjukkan *gap* solusi SCIP.

Tabel VII
STATISTIK BIAYA DARI 30 *Run* KONFIGURASI TERBAIK SETIAP ALGORITMA DAN *Gap* TERHADAP OPTIMUM *Brute-Force*

n	Algoritma	Terbaik	Rata-rata	Simpangan baku	Terburuk	*Gap* rata-rata (%)	*Gap* terburuk (%)
13	GA	5.223.596,12	5.223.974,73	958,79	5.226.720,98	0,0072	0,0598
	BPSO	5.223.596,12	5.223.722,10	503,97	5.226.249,39	0,0024	0,0508
	Random	5.223.596,12	5.223.996,89	573,86	5.225.809,41	0,0077	0,0424
16	GA	5.801.121,21	5.802.399,81	1.131,78	5.805.242,11	0,0220	0,0710
	BPSO	5.801.121,21	5.802.163,26	1.197,45	5.805.146,57	0,0180	0,0694
	Random	5.801.121,21	5.805.164,48	1.378,01	5.808.028,91	0,0697	0,1191
18	GA	5.962.911,99	5.963.414,78	508,44	5.965.061,05	0,0084	0,0360
	BPSO	5.962.911,99	5.963.512,82	621,08	5.965.833,89	0,0101	0,0490
	Random	5.963.302,42	5.965.369,04	1.241,58	5.969.033,44	0,0412	0,1027
19	GA	6.112.960,59	6.115.027,31	984,31	6.117.468,33	0,0338	0,0737
	BPSO	6.112.960,59	6.114.840,69	1.154,43	6.117.468,33	0,0308	0,0737
	Random	6.113.101,33	6.118.758,63	2.683,04	6.125.572,41	0,0948	0,2063

Catatan: *gap* terbaik bernilai 0 kecuali untuk *random search* pada $n = 18$ (0,0065%) dan $n = 19$ (0,0023%). Pada instans 3 dan 9, ketiga algoritma mencapai optimum pada seluruh *run*.

Tabel VIII
MEDIAN *Gap* DAN NILAI p UJI MANN-WHITNEY U UNTUK SETIAP PASANGAN ALGORITMA

n	Median GA / BPSO / RS (%)	GA vs BPSO	BPSO vs RS	GA vs RS
13	0 / 0 / 0,0026	0,409	**0,001**	**0,029**
16	0,0264 / 0,0132 / 0,0742	0,305	$<$ **0,001**	$<$ **0,001**
18	0,0041 / 0,0065 / 0,0403	0,623	$<$ **0,001**	$<$ **0,001**
19	0,0332 / 0,0302 / 0,0882	0,666	$<$ **0,001**	$<$ **0,001**

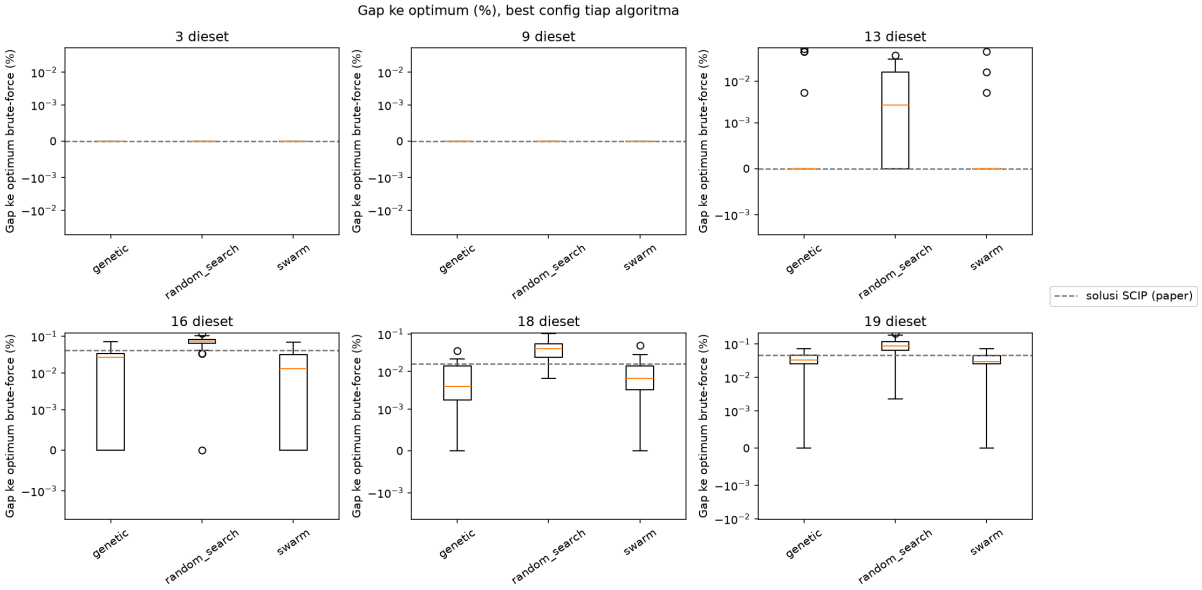
RS: *random search*. Nilai p bercetak tebal signifikan pada $\alpha = 0{,}05$.

ritma tersebut secara signifikan lebih baik daripada *random search* pada instans 13 hingga 19 pada $\alpha = 0{,}05$.

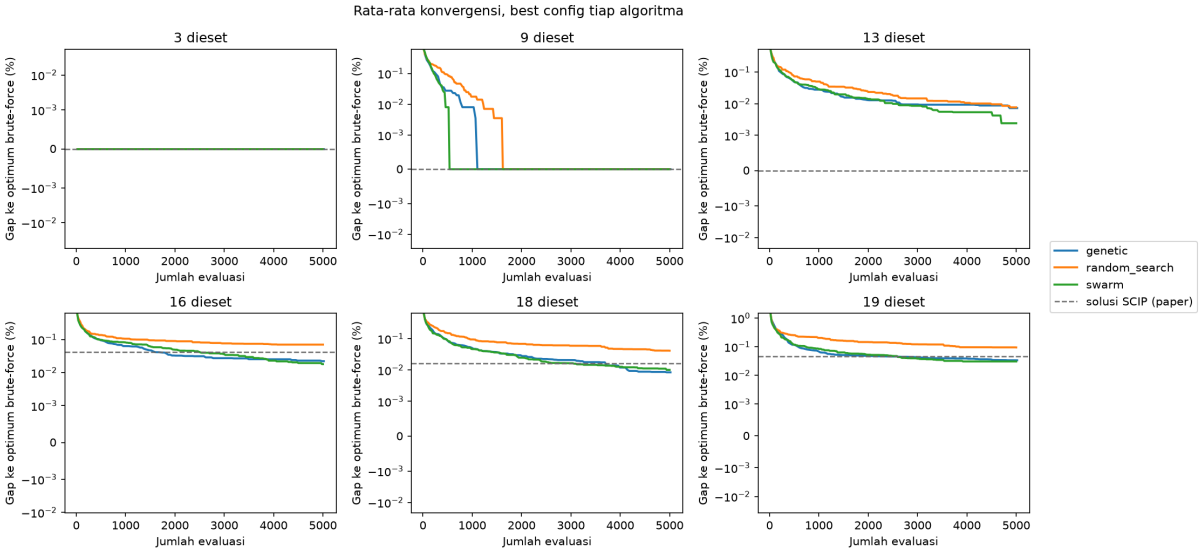
D. Konvergensi, Waktu Eksekusi, dan Skalabilitas

Gambar 7 menunjukkan bahwa kurva GA dan BPSO menurun lebih cepat daripada *random search* sejak ratusan evaluasi pertama. Pada instans 9, solusi terbaik ditemukan rata-rata setelah 239 evaluasi oleh BPSO, 301 evaluasi oleh GA, dan 567 evaluasi oleh *random search*. Pada instans 16, 18, dan 19, solusi terbaik rata-rata baru ditemukan setelah 1.800 hingga 3.000 evaluasi, dan median *gap* BPSO pada instans 16 serta GA pada instans 18 masih menurun setelah evaluasi ke-4.000. Kurva rata-rata GA dan BPSO berada di bawah garis solusi SCIP mulai sekitar evaluasi ke-1.700 hingga ke-3.000 pada instans 16 dan 19.

Gambar 8 merangkum metrik evaluasi konfigurasi terbaik setiap algoritma terhadap jumlah *dieset*. Waktu eksekusi rata-rata per *run* adalah 0,081 detik untuk GA, 0,018 detik untuk BPSO, dan 0,013 detik untuk *random search*. Persentase *run* yang mencapai optimum menurun dari 100% pada instans 3 dan 9 menjadi paling tinggi 13,3% pada instans 19. *Gap random search* meningkat paling cepat, yaitu dari 0,0077%



Gambar 6. Sebaran *gap* dari 30 *run* konfigurasi terbaik setiap algoritma pada keenam instans. Garis putus-putus menunjukkan *gap* solusi SCIP.



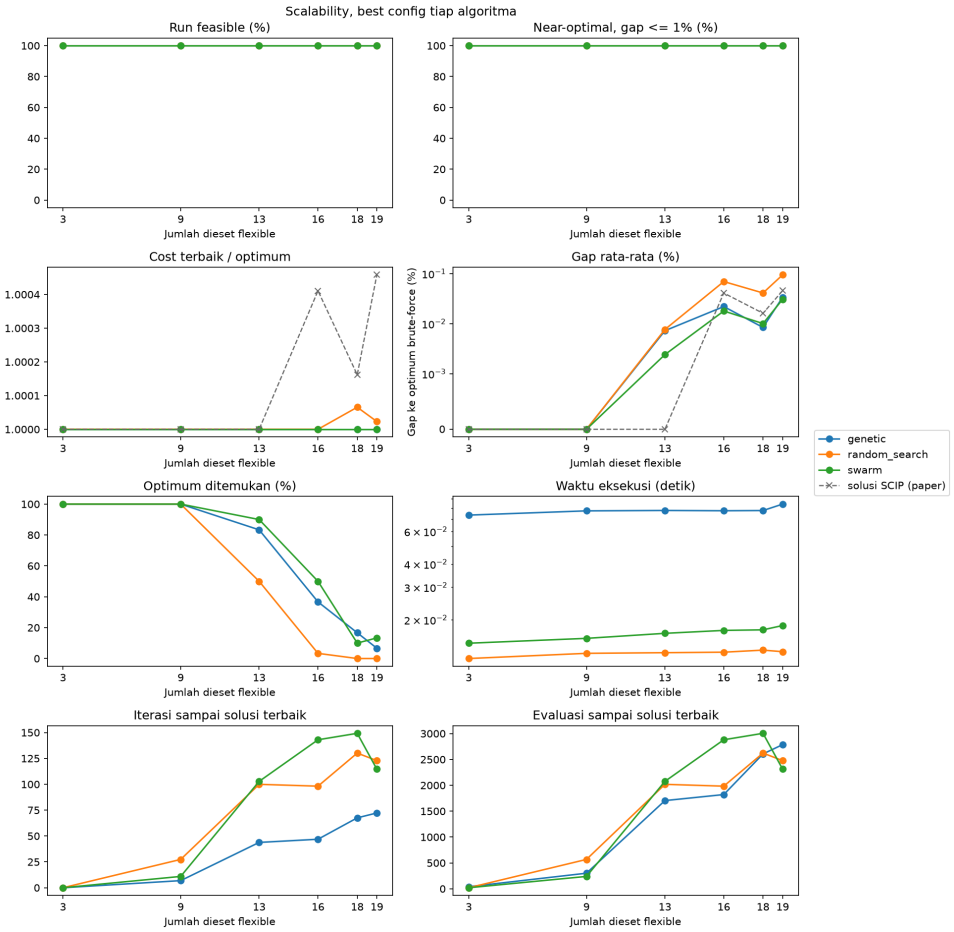
Gambar 7. Kurva konvergensi, yaitu rata-rata *gap* solusi terbaik dari 30 *run* terhadap jumlah evaluasi untuk konfigurasi terbaik setiap algoritma. Garis putus-putus menunjukkan *gap* solusi SCIP.

pada instans 13 menjadi 0,0948% pada instans 19, sedangkan *gap* GA dan BPSO tetap berada di bawah 0,035%.

IV. Diskusi

A. Kelayakan Solusi dan Besaran Gap

Proporsi solusi layak pada *dataset* ini relatif besar, yaitu 35% hingga 44% dari seluruh kombinasi pada instans 9 hingga 19 (Tabel I). Dengan anggaran 5.000 evaluasi, setiap algoritma, termasuk *random search*, menjumpai ribuan solusi layak, sementara suku indikator pada (6) menjamin bahwa solusi layak selalu lebih diutamakan. Nilai *gap* yang kecil tidak berarti bahwa persoalan ini mudah. Pada instans 9 hingga 19, solusi layak terburuk memiliki *gap* 2,9% hingga 10%, sedangkan solusi termurah tanpa kendala kapasitas berada 3% hingga 5% di bawah optimum. Hal ini menunjukkan bahwa kendala kapasitas aktif dan menentukan letak solusi optimum. *Gap* yang kecil disebabkan oleh biaya satu *dieset* yang relatif kecil terhadap total biaya, sehingga solusi yang hanya berbeda pada satu atau dua *dieset* sudah memiliki *gap* di bawah 0,1%. Pada skala ini, kriteria *near-optimal* 1% dari makalah [3] tidak mampu membedakan kinerja algoritma. Oleh karena itu, perbandingan dalam penelitian ini difokuskan pada *gap* rata-rata, persentase pencapaian optimum, dan uji signifikansi.



Gambar 8. Skalabilitas konfigurasi terbaik setiap algoritma terhadap jumlah *dieset* fleksibel: persentase *run* layak dan *near-optimal*, rasio biaya terbaik terhadap optimum, *gap* rata-rata, persentase pencapaian optimum, waktu eksekusi, serta jumlah iterasi dan evaluasi hingga solusi terbaik ditemukan.

B. Keunggulan GA dan BPSO terhadap Random Search

Anggaran 5.000 evaluasi mencakup seluruh ruang pencarian pada instans 3 dan 9 serta sekitar 61% ruang pencarian pada instans 13, tetapi hanya 7,6%, 1,9%, dan 0,95% pada instans 16, 18, dan 19. Pada instans berukuran kecil, *random search* hampir pasti menemukan optimum sehingga perbedaannya dengan algoritma lain kecil atau tidak signifikan. Seiring bertambahnya ukuran ruang pencarian, *random search* hanya bergantung pada peluang, sedangkan GA dan BPSO memusatkan pencarian di sekitar solusi yang baik. Penjelasan ini sejalan dengan kurva konvergensi yang telah terpisah sejak ratusan evaluasi pertama dan dengan peningkatan *gap random search* yang paling cepat terhadap n.

C. Kesetaraan Kinerja GA dan BPSO

Kedua algoritma menggunakan representasi dan fungsi *fitness* yang sama. Keduanya juga menggabungkan mekanisme eksploitasi, yaitu elitisme dan seleksi pada GA serta tarikan menuju p_i dan g pada BPSO, dengan mekanisme eksplorasi stokastik, yaitu mutasi pada GA dan pengambilan sampel posisi berdasarkan peluang pada BPSO. Pada ruang pencarian dengan dimensi maksimum 19, kedua mekanisme tersebut cukup untuk mencapai daerah yang mendekati optimal, dan selisih kinerja yang tersisa lebih kecil daripada variasi antar-*seed*. Dengan demikian, kesimpulan yang didukung oleh data adalah bahwa kedua algoritma tidak berbeda secara signifikan pada *dataset* ini, bukan bahwa salah satu algoritma lebih unggul.

D. Tingkat Kesulitan Instans

Tingkat kesulitan instans tidak semata-mata ditentukan oleh n. Hasil enumerasi menunjukkan bahwa instans 18 memiliki 68 solusi dengan *gap* maksimum 0,05%, sedangkan instans 16 hanya memiliki 6 solusi dan instans 19 memiliki 19 solusi. Semakin banyak solusi yang mendekati optimal, semakin besar peluang algoritma menemukan salah satu di antaranya dalam anggaran evaluasi yang sama. Hal ini menjelaskan mengapa instans 18 lebih mudah diselesaikan daripada instans 16 dan 19.

E. Interpretasi Analisis Sensitivitas

- Pada GA, populasi yang lebih besar mempertahankan keragaman genetik lebih lama. Seleksi *tournament* berukuran 3 memberikan tekanan seleksi terkuat di antara ketiga metode seleksi sehingga populasi pada konfigurasi

bawaan diduga mengalami konvergensi prematur. *Rank selection* dan *roulette wheel* menurunkan tekanan seleksi dengan efek yang serupa dengan penambahan ukuran populasi.
- Pada BPSO, *swarm* berukuran 20 memperoleh 249 iterasi, atau dua kali lipat jumlah iterasi *swarm* berukuran 40. Karena posisi terbaik *swarm* tercapai dengan cepat, BPSO lebih diuntungkan oleh jumlah iterasi pembaruan yang lebih banyak daripada oleh jumlah partikel yang lebih banyak. Kecenderungan ini berlawanan dengan GA, yang lebih diuntungkan oleh populasi yang besar.
- Mutasi *swap* hanya menukar nilai dua gen sehingga jumlah *dieset* pada setiap mesin tidak berubah. Karena solusi optimum terletak di batas kapasitas, pencarian memerlukan perpindahan *dieset* dari satu mesin ke mesin lain, yang tidak dapat dilakukan oleh mutasi *swap*, tetapi dapat dilakukan oleh mutasi *bit-flip*.
- *Uniform crossover* lebih baik daripada *one-point* dan *two-point* karena urutan *dieset* pada kromosom tidak bermakna sehingga tidak terdapat blok gen bersebelahan yang perlu dipertahankan.

F. Perbandingan dengan Solusi SCIP

Pada instans 16, 18, dan 19, sebagian besar *run* GA dan BPSO (70% hingga 93%) menghasilkan biaya yang lebih rendah daripada solusi SCIP pada makalah [3]. Hasil ini tidak dapat diartikan bahwa metaheuristik mengungguli *solver* eksak. Solusi SCIP merupakan solusi optimal bagi model dengan kapasitas yang dibulatkan ke bawah dan beban yang dibulatkan ke atas, sehingga ruang solusi layaknya lebih sempit daripada model tanpa pembulatan yang digunakan dalam penelitian ini.

G. Keterbatasan Penelitian

1) Ukuran instans relatif kecil (maksimum 2^{19} kombinasi) sehingga optimum masih dapat dihitung dengan *brute-force* dalam hitungan detik. Keunggulan metaheuristik pada persoalan nyata berskala lebih besar belum teruji.
2) Seluruh instans hanya melibatkan dua mesin sehingga representasi berbentuk vektor biner. BPSO perlu diperluas agar dapat menangani lebih dari dua pilihan mesin.
3) *Baseline* yang digunakan hanya *random search*. *Simulated annealing*, yang juga digunakan sebagai referensi pada makalah [3], dan *hill climbing* belum dibandingkan.
4) Analisis sensitivitas dilakukan secara *one-factor-at-a-time* sehingga interaksi antarparameter tidak terukur. Selain itu, pemilihan dan perbandingan konfigurasi terbaik menggunakan *seed* yang sama sehingga hasil konfigurasi terbaik berpotensi sedikit optimistis.
5) Uji signifikansi belum dikoreksi untuk perbandingan berganda. Dengan koreksi Bonferroni terhadap 12 uji ($\alpha = 0{,}0042$), perbandingan GA dan *random search* pada instans 13 ($p = 0{,}029$) tidak lagi signifikan, sedangkan kesimpulan lainnya tidak berubah.
6) Pada instans 16 dan 19, kurva konvergensi masih menurun pada akhir anggaran evaluasi sehingga hasil dapat berubah apabila anggaran diperbesar.
7) Waktu eksekusi bergantung pada perangkat keras dan cara implementasi, khususnya vektorisasi NumPy, sehingga hanya bermakna sebagai perbandingan relatif.

V. KESIMPULAN

Persoalan alokasi *dieset* ke mesin pres pada *dataset Path Matters* dapat dimodelkan sebagai persoalan optimasi biner dengan representasi satu gen untuk setiap *dieset* fleksibel, yang secara otomatis memenuhi kendala penugasan, serta fungsi penalti kuadratik yang dilengkapi suku indikator untuk menangani kendala kapasitas. Dengan anggaran 5.000 evaluasi, GA dan BPSO menghasilkan solusi layak pada seluruh *run*, dengan *gap* rata-rata konfigurasi terbaik di bawah 0,035% dan optimum ditemukan sekurang-kurangnya satu kali pada setiap instans. Kedua algoritma secara signifikan lebih baik daripada *random search* pada instans 13 hingga 19, dan keunggulan tersebut tetap signifikan setelah koreksi Bonferroni pada instans 16 hingga 19 serta semakin besar seiring bertambahnya jumlah *dieset*. GA dan BPSO tidak berbeda secara signifikan, tetapi BPSO sekitar 4,5 kali lebih cepat per *run*. Ukuran populasi atau *swarm* merupakan parameter yang paling berpengaruh: GA diuntungkan oleh populasi yang lebih besar, sedangkan BPSO diuntungkan oleh *swarm* yang lebih kecil dengan jumlah iterasi yang lebih banyak.

Penelitian selanjutnya dapat diarahkan pada pengujian instans berukuran lebih besar dengan lebih dari dua mesin, penambahan *baseline* berupa *simulated annealing* dan *hill climbing*, penambahan *local search* di sekitar batas kapasitas (*memetic algorithm*), serta penyetelan parameter menggunakan *seed* yang terpisah dari *seed* evaluasi.

KONTRIBUSI PENULIS

F. Sucahyo melakukan pra-pemrosesan kapasitas sisa dan optimum *brute-force*, mengimplementasikan BPSO dan *baseline random search*, melakukan eksperimen dan analisis BPSO, uji Mann-Whitney U, serta menyiapkan konfigurasi eksperimen dan reproduksi. N. Najmudin menyiapkan proyek dan *dataset*, merancang representasi persoalan dan kelas dasar algoritma, mengimplementasikan algoritma genetika dan metrik evaluasi, menjalankan seluruh eksperimen, serta melakukan analisis GA dan perbandingan antaralgoritma. Kedua penulis menulis dan merevisi naskah.

PERNYATAAN PENGGUNAAN ALAT BANTU AI

Penulis memanfaatkan Claude AI sebagai alat bantu dalam penyusunan outline naskah, pembuatan diagram alir, serta penulisan sebagian kode pendukung, seperti skrip visualisasi. Perumusan persoalan, representasi, rancangan eksperimen, dan interpretasi hasil sepenuhnya dilakukan oleh penulis. Seluruh kode yang digunakan dalam penelitian ini telah ditinjau, diuji, dan diverifikasi oleh penulis, yang bertanggung jawab penuh atas isi artikel ini.

PUSTAKA

[1] G. T. Ross and R. M. Soland, "A branch and bound algorithm for the generalized assignment problem," *Math. Program.*, vol. 8, pp. 91–103, 1975.

[2] P. C. Chu and J. E. Beasley, "A genetic algorithm for the generalised assignment problem," *Comput. Oper. Res.*, vol. 24, no. 1, pp. 17–23, 1997.

[3] L. Schmidbauer, C. A. Riofrío, F. Heinrich, V. Junk, U. Schwenk, T. Husslein, and W. Mauerer, "Path matters: Industrial data meet quantum optimization," in *Proc. IEEE Int. Conf. Quantum Computing and Engineering (QCE)*, 2025, arXiv:2504.16607.

[4] L. Schmidbauer *et al.*, "Path matters: Industrial data meet quantum optimization (reproduction package and dataset)," Zenodo, 2025, doi: 10.5281/zenodo.16269046. [Online]. Available: https://zenodo.org/records/16269046

[5] H. Safi, K. Wintersperger, O. von Sicard, C. Niedermeier, and W. Mauerer, "A reality check on quantum optimisation: Evidence from an industrial case study," 2026, arXiv:2607.13325.

[6] S. Thelen and W. Mauerer, "From constraint to code: DQI-Kit – A software framework for decoded quantum interferometry," 2026, arXiv:2605.16955.

[7] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA, USA: Addison-Wesley, 1989.

[8] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Networks (ICNN)*, 1995, pp. 1942–1948.

[9] J. Kennedy and R. C. Eberhart, "A discrete binary version of the particle swarm algorithm," in *Proc. IEEE Int. Conf. Systems, Man, and Cybernetics*, 1997, pp. 4104–4108.

[10] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *Proc. IEEE Int. Conf. Evolutionary Computation*, 1998, pp. 69–73.

[11] S. Mirjalili and A. Lewis, "S-shaped versus V-shaped transfer functions for binary particle swarm optimization," *Swarm Evol. Comput.*, vol. 9, pp. 1–14, 2013.

[12] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Ann. Math. Statist.*, vol. 18, no. 1, pp. 50–60, 1947.

[13] F. Sucahyo and N. Najmudin, "AI5103-28426004-13523144: Optimasi alokasi dieset dengan GA dan BPSO," GitHub repository, 2026. [Online]. Available: https://github.com/nazihnajmudin/AI5103-28426004-13523144